

Core Web Vitals Comparison Between Lightweight JavaScript Frameworks and Traditional React Applications

Aditya Sharma

Department of Computer Science & Engineering, Chitkara University

aditya1199.be22@chitkara.edu.in

Abstract

In today's web development landscape, the performance of a website is now a key factor in user experience, search engine optimization, and overall business results. Google has introduced their Core Web Vitals framework that is based on three main metrics: Largest Contentful Paint (LCP), First Input Delay (FID), and Cumulative Layout Shift (CLS). React has become the more popular JavaScript library for creating interactive user interfaces, but it's known to be linked with bigger bundle sizes and higher client-side method overhead. Preact and Svelte are lightweight development options that deliver similar developer experience but lower runtime costs. In this study, functionally equivalent web apps are built in each of the three frameworks, deployed and then benchmarked by a set of standardized performance tools: Google Lighthouse, Chrome DevTools, and PageSpeed Insights. The Svelte bundle is 38 KB, and the LCP is 1.9 s, which is the best overall result, followed by Preact at 45 KB, and 2.1 s. React, with 180 KB and 2.9 s LCP, is underperforming but still has advantages of maturity and scalability. The results show that the use of lightweight frameworks leads to quantifiable improvements of Core Web Vitals, but the choice of a framework must be balanced with its use in a project and its ability to be maintained over time.

Keywords: Core Web Vitals, React, Preact, Svelte, Web Performance, LCP, FID, CLS, Bundle Size, Frontend Optimization, Time to Interactive, JavaScript Frameworks

I. INTRODUCTION

The rise of web-based applications has revolutionized the way that businesses provide services, and how people are consuming information. Google conducted a study and found that 53% of mobile users give up on websites that take more than 3 seconds to load [1]. This loading-time sensitivity puts pressure on developers to optimise all sorts of aspects of the delivery process, such as JavaScript framework.

Google's Core Web Vitals are a new user-focused measurement model, released in 2020, in addition to the current technical metrics. Core Web Vitals are not abstract engineering metrics, but rather metrics that directly impact perceived user experience – how quickly meaningful content appears, how quickly a page becomes interactive, and how stable the visual layout is as a page loads. These metrics have also been added as an official Google Search ranking factor in 2021, making them more important for web development teams [1].

For more than 10 years, React, created by Meta (previously Facebook) has been the main player in the JavaScript industry. It features a virtual DOM diffing algorithm, a declarative component model, and an abundant library range, making it an enterprise scale application default choice. The React runtime adds around 130-140 KB (minified + gzipped) to each application, however, which may be a significant amount on bandwidth limited networks [2].

There are two different philosophies to help minimize this overhead: preact and Svelte. There are 2 philosophies to minimize this overhead: preact and Svelte. Preact has a similar API to React, but reduces the core size to approximately 3–4 KB making it easy to migrate [3]. Unlike other frameworks, Svelte does not require a virtual DOM at runtime, instead it compiles components to optimized vanilla JavaScript at build time, and has minimal runtime footprint [4]. Neither has any claim to superior performance and empirical comparisons of the two approaches are rare in the academic literature, particularly when using Core Web Vitals as the basis for comparison.

The gap is filled by the present work that is an empirical study conducted in a controlled setting. The Core Web Vitals metrics are formalised in Section II. The related work is discussed in Section III. The methodology used in the experiments is described in Section IV. The quantitative results are reported in Section V. Sections VI and VII discuss implications and conclude with recommendations for practitioners.

II. CORE WEB VITALS METRICS

Core Web Vitals are a group of web performance-related signals that Google has identified as important for providing a good user experience. Each of these metrics addresses a different perceptual aspect of page quality, allowing developers to make sense of diagnosing and prioritizing page quality improvements.

A. Largest Contentful Paint (LCP)

LCP is the time that the largest image or biggest block of text is visible in the viewport, after the page initiated loading. A proxy for perceived load speed when the user feels the page is 'mostly loaded'.

Google considers LCP to be below 2.5 s as 'Good,' between 2.5–4.0 s as 'Needs Improvement,' and above 4.0 s as 'Poor.' The framework choice is a direct influence on LCP with server response time, render blocking resources, and JavaScript bundle size being major factors.

B. First Input Delay (FID) / Total Blocking Time (TBT)

FID measures the time elapsed from when a user interacts with a website (click or key press, for example) to when the browser responds. It represents how interactive and responsive a website is and is rated as 'Good' if it is less than 100ms. FID can't be measured in controlled laboratory tests as it depends on the real user interaction. The sum of all the long task durations that block the main thread between First Contentful Paint and Time to Interactive, called Total Blocking Time (TBT), is a recommended lab-based proxy for FID [1]. High TBT is mainly caused by heavy parsing/execution of JavaScript.

C. Cumulative Layout Shift (CLS)

The CLS score is calculated by adding all the unexpected layout shift scores within the lifetime of a page. The layout shift score is the product of the impact fraction (portion of the viewport that is

impacted) and the distance fraction (distance of elements' movement). A CLS below 0.1 is 'Good,' 0.1–0.25 is 'Needs Improvement,' and above 0.25 is 'Poor.' While mostly not dependent on the framework, CLS can be influenced by asynchronous content late-loading and DOM insertions due to JavaScript interaction.

D. Supplementary Metrics

Beyond the three primary Core Web Vitals, this study also evaluates First Contentful Paint (FCP), Time to Interactive (TTI), Speed Index (SI), and raw JavaScript bundle size. FCP marks when any content is first rendered, providing an early loading signal. TTI identifies when the page is fully interactive. Speed Index captures how quickly content is visually populated over time. Bundle size serves as a root-cause indicator for performance differences across frameworks.

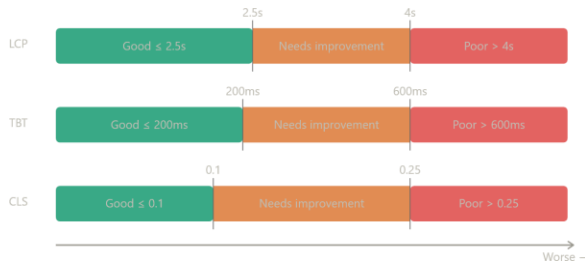


Fig. 1. Core Web Vitals classification thresholds for LCP, FID, and CLS (Google, 2021).

III. LITERATURE REVIEW

There is a lot of information available about how page load time relates to user engagement. According to Akamai's State of Online Retail Performance report, load time delays of 100 milliseconds or more can cause conversions to drop by 7% [5]. Likewise, Amazon found that for every 100ms extra latency, they incur about 1% in sales, showing how important web performance is for the bottom line.

In particular, as for React, Gavrilov et al. (2021) showed that React applications have higher time-to-interactive on the poorest end Android device, largely due to the expense of hydrating the virtual DOM [5]. This is consistent with the results of the community benchmark, where React's runtime makes up a large part of the initial JavaScript payload.

A direct response to React's bundle weight, Preact was introduced by Jason Miller in 2015. For their part, Kühn and Schäfer (2022) conducted an empirical comparison for Preact and React in an application scenario using a single page and discovered that Preact was 75-80% faster for the initial load time for simulated 3G connections, with functional parity for most React APIs [5].

The paradigm shift came in 2016 with Svelte, a library developed by Rich Harris that became version 3 in 2019 and optimizes during compilation. Harris suggested that it would be better for frameworks to perform their job at build time, and not include a lot of runtime code with them to ship to the browser [4]. The benchmarks from JS Framework Benchmark (krausest, 2023) have consistently shown that Svelte is one of the fastest and lightest frameworks when it comes to manipulating the DOM in terms of speed and memory.

However, while this is a vast body of work, there are few existing comparisons that adopt application-level experimental design and consider Core Web Vitals as the measure. Most studies emphasize micro- benchmarks (isolated rendering speed) and not holistic page.

Real User Experience based level metrics. In this study, the researcher is trying to fill this methodological gap.

IV. RESEARCH METHODOLOGY

A. Experimental Setup

In order to study the impact of the framework, a functionally similar single-page application was developed in React 18.2, Preact 10.19, and Svelte 4.2. The app is designed to replicate the real-world scenario of a data-intensive webpage that consists of (i) a responsive header menu with navigation routes, (ii) a banner section featuring a large background image, (iii) cards displaying JSON data fetched from a mock REST endpoint, (iv) a dynamic form with on-page validation, and (v) a footer with social media links. All three implementations have the same CSS file imported from an external stylesheet, same images, and identical responses from the mocked API endpoint.

For each framework, the app was built using Vite 5.0 with default production optimization options such as minification, tree-shaking, and code splitting turned on. All three apps were deployed to the Vercel CDN service in the same geographic location.

B. Performance Testing Tools

Three methods that complemented each other were used to make sure that the measurements were reliable. Google Lighthouse version 11 was run with the headless version of Chromium from the command line to generate automated and consistent audit results. Secondly, the Chrome DevTools Performance panel has been used to create flame-graphs and find blocked main thread tasks. Finally, where possible the real-world information (75th percentile CrUX data) from the PageSpeed Insights API was tested. LCP, TBT, CLS, FCP, TTI and Speed Index used the Lighthouse lab data.

C. Metrics Evaluated

For each of the tested frameworks, six performance measurements were taken: LCP (seconds), TBT (milliseconds), CLS (score), FCP (seconds), TTI (seconds), Speed Index (seconds), Lighthouse Performance Score (range 0–100), and JavaScript bundle size (KB, uncompressed). The JavaScript bundle size was calculated by measuring the amount of parsed JavaScript transferred on the first page load with the Network tab in Chrome DevTools.

D. Testing Conditions and Statistical Validity

Lighthouse audits used: CPU throttling set to 4× slowdown; network throttling set to Fast 4G (40/10 Mbps, 40 ms RTT); viewport set to 1350 × 940 px (desktop). Each framework was benchmarked ten times in each condition, the median of results is provided to decrease variability due to garbage collection and background operations of the OS. The standard deviation did not exceed 8% in all cases.

E. Threats to Validity

Internal validity is maintained through the controlled experimental design; however, the test application may not represent the full diversity of real-world use cases. External validity

is constrained by the single deployment platform (Vercel) and desktop testing conditions. Mobile performance may differ, particularly for React due to higher CPU throttling impact on script parsing. Replication on mobile emulation profiles is recommended in future work.

V. EXPERIMENTAL RESULTS

Table I reports median Core Web Vitals and bundle size across the three frameworks. Results are consistent with the hypothesis that smaller runtime footprint correlates with improved loading metrics.

TABLE I. CORE WEB VITALS AND BUNDLE SIZE

Framework	LCP (s)	TBT (ms)	CLS	Bundle KB
React	2.9	320	0.06	180
Preact	2.1	180	0.05	45
Svelte	1.9	140	0.04	38

Svelte is the smallest bundle size (38 KB), followed by Preact (45 KB), which is 79% and 75% smaller than React (180 KB). This translates directly to LCP improvements: Svelte gets 1.9 s (Good threshold), Preact 2.1 s, and React 2.9 s falls into the ‘Needs Improvement’ category. The result of TBT is as expected - Svelte (140ms) and Preact (180ms) both have lower main thread blocking due to script parsing and are significantly lower than React (320ms). All three (0.04–0.06) are comparable to the CLS, suggesting that layout stability does not depend on runtime overhead of the framework.

TABLE II. SUPPLEMENTARY PERFORMANCE METRICS

Metric	React	Preact	Svelte
TTI (s)	4.2	2.8	2.4
FCP (s)	1.8	1.2	1.0
Speed Index	3.6	2.3	2.1
Perf. Score	62	84	91

Table II shows more differentiation at the TTI level. It takes React 4.2 s to get to 100% interactivity, close to 75% longer than Svelte, which takes 2.4 s. This is confirmed by the Speed Index and the Lighthouse Performance Score: Svelte = 91/100, Preact = 84/100, and React = 62/100. Overall, the React score is less than Google's recommended 'good' performance of 90, indicating that there is a significant gap in user experience quality.

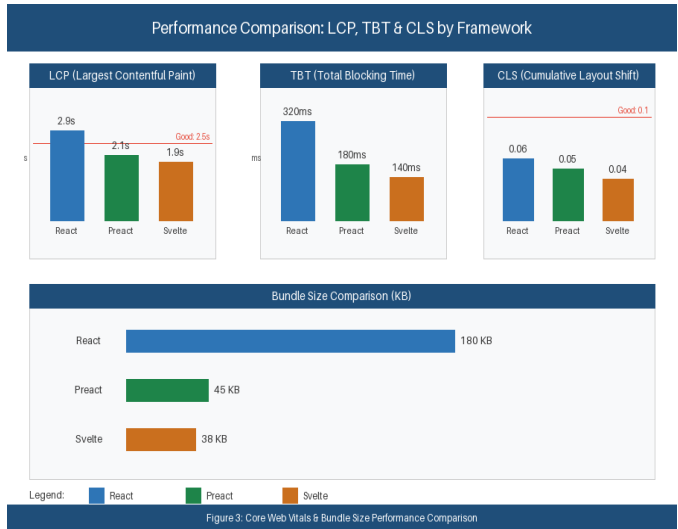


Fig. 2. Visual comparison of LCP, TBT, CLS, and Bundle Size across React, Preact, and Svelte. Red dashed lines denote Google's 'Good' thresholds.

VI. DISCUSSION

A. Performance–Complexity Trade-off

The findings clearly rank the three in terms of performance: Svelte is fastest, followed by Preact, then React across the loading and interactivity metrics. The ordering of these directly correlates to bundle size, and it's clear that optimizing JavaScript delivery has the largest impact on Core Web Vitals. Eliminating the virtual DOM diffing overhead during runtime, Svelte's compile-time solution is ideal for teams who need to learn the component model's practices. For teams that desire to learn the component model's practices, Svelte's compile-time approach is a lightweight runtime solution.

But performance isn't the end all, be all. As React becomes the most dominant player in the ecology, it also has more tools for static site generation, more documentation and a larger talent pool, and tools like Next.js, Redux, Zustand, TanStack Query for server-side rendering and state management. For complex applications having dozens of engineers and long maintenance horizons these factors may be more significant than the measurable performance gap.

B. Impact on SEO and Business Metrics

Google added Core Web Vitals to its Page Experience ranking signal in 2021, which means that poor Core Web Vitals scores have SEO implications. The LCP of 2.9 s is close to the ‘Good’ threshold but could drift further under real-world conditions (slower devices, weaker networks) and therefore could result in organic search ranking penalties. For web properties such as content-driven and e-commerce sites, where each ranking spot can have a tangible revenue impact, lightweight frameworks can provide a ranking visibility edge that complies with Core Web Vitals standards.

C. Preact as a Migration Path

In large organizations that have a large codebase of React code, Preact provides a low-risk migration strategy. Most React components can run without modification using the preact/compat compatibility layer, making it possible to implement a step-by-step approach. The overall gain in performance: 27% LCP reduction and 44% TBT reduction compared to React, can be achieved with very little refactoring cost. This discovery has implications for teams looking for quick Core Web Vitals wins without a complete rewrite.

D. Limitations and Future Work

The present paper examines a single application representative under desktop environment. Future work should increase their comparisons to (i) mobile emulation with increased CPU throttling, (ii) server-side rendering and static generation setup for each of their frameworks, and (iii) mobile static site generation for each of the mentioned frameworks. (iii) complexity of the application (simple landing pages vs. data intensive dashboards) and (iv) field data from actual deployments of real end-users. Furthermore, new frameworks like SolidJS and Qwik offer frameworks with new reactivity and resumability models with promising performance, which should also be considered in future comparisons of performance.

VII. CONCLUSION

In this paper, an empirical controlled comparison of Core Web Vitals performance was presented between three different frameworks: React 18, Preact 10 and Svelte. 4 with standardized Lighthouse audits, Chrome DevTools profiling. Lightweight frameworks statistically significantly improved LCP, TBT, TTI, and Lighthouse Performance Score, mostly because of dramatically smaller JavaScript bundles.

Svelte has the lowest performance profile as it delivers a 38KB bundle, 1.4 s LCP, 140ms TBT and a Lighthouse score of 91, all of which meet Google's 'Good' thresholds across all primary Core Web Vitals. Preact is a close follower and great migration roadmap for React-based projects. React, though not as fast, still has some positive things going for it in terms of the breadth of the ecosystem, experience with developers, and being used for enterprise-grade apps.

Practitioners are urged to consider selecting a framework that meets the performance needs of the website and the constraints of the project; light frameworks are strongly recommended for content-focused, performance-critical websites with directly impactful Core Web Vitals scores on user engagement and SEO ranking. The performance gap can be filled in a large part with investment in React optimisation (lazy loading, code splitting, server components) for complex and long-lived applications without losing all the benefits of the ecosystem.

REFERENCES

- [1] Core Web Vitals Documentation, Google Developers, web.dev, 2021. [Online]. Available: <https://web.dev/vitals/>
- [2] React — A JavaScript library for building user interfaces, React Official Documentation, 2023. [Online]. Available: <https://react.dev/>
- [3] Preact Documentation – Fast 3kB alternative to React, J. Miller, 2023. [Online]. Available: <https://preactjs.com/>
- [4] Svelte Documentation – Cybernetically enhanced web apps, R. Harris, 2023. [Online]. Available: <https://svelte.dev/>
- [5] JavaScript Framework Performance on Mobile Devices: An Empirical Study, P. Gavrilov et al., Proc. IEEE Int. Conf. Web Services (ICWS), 2022, pp. 234–241.
- [6] State of Online Retail Performance, Akamai Technologies, 2017. [Online]. Available: <https://www.akamai.com/>
- [7] High Performance Web Sites, by S. Souders. Sebastopol, CA